



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

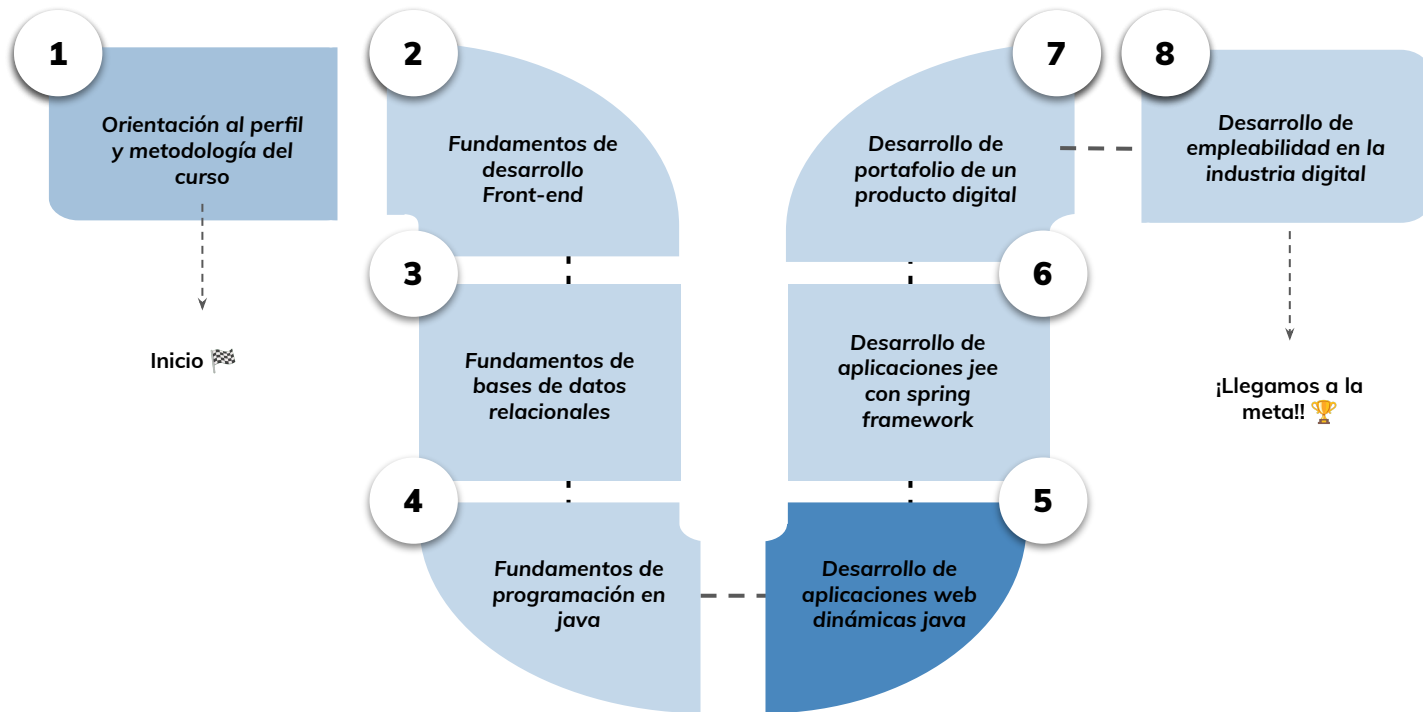


› Java server pages - Parte I

Aprendizaje Esperado 2: Implementar capa de vista de una aplicación web dinámica utilizando java server pages de acuerdo a las especificaciones entregadas.

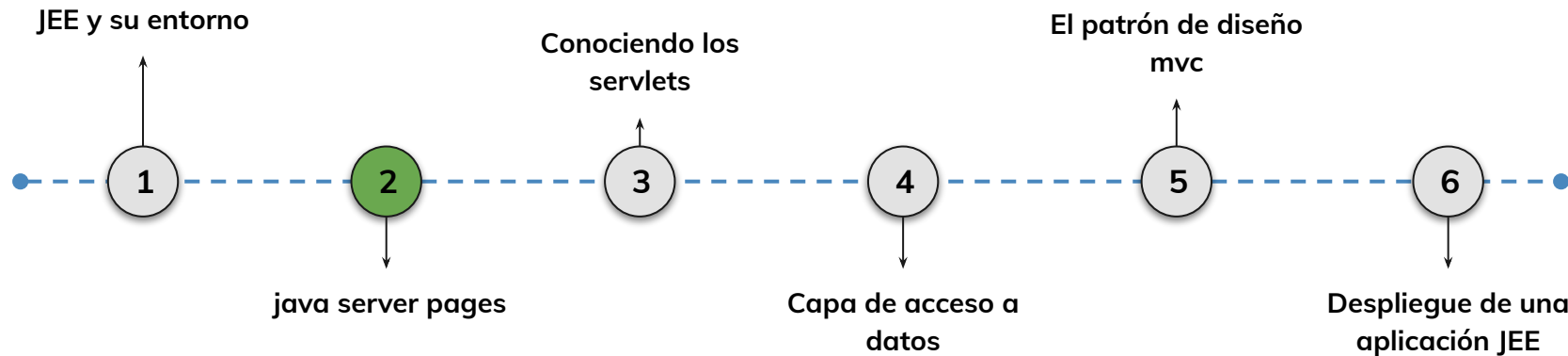
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



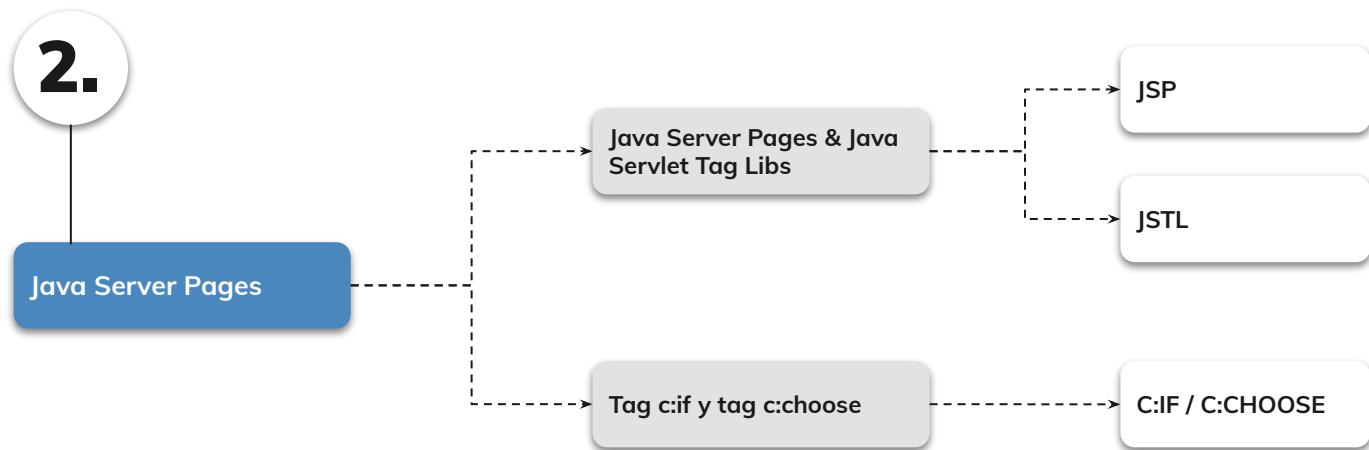
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender el concepto e implementación de las Java Server Pages
- Aprender a utilizar los tags JSTL
- Reconocer la implementación de la etiqueta `c:out`
- Comprender el uso de las etiquetas `c:if` y `c:choose`



Rompehielo 🧊

Respondan en el chat o levantando la mano 🙋

¿Creen que es posible escribir código Java en archivos HTML? ¿Por qué?

¿Qué les parece el siguiente código? ¿Podría funcionar? ¿Por qué?

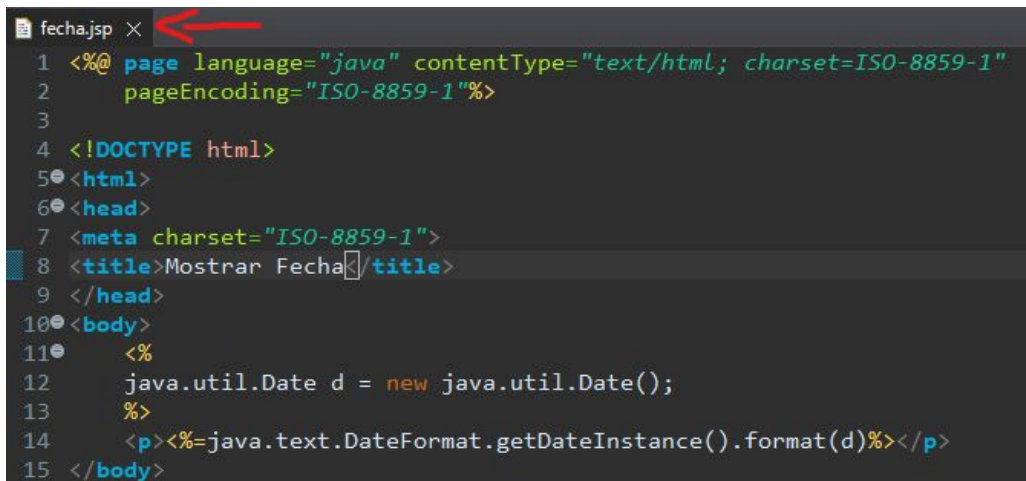
```
3 <html>
4 <body>
5 <%
9 <p>Hola <%= nombre %>, veo que tienes <%= edad %> años.</p>
10
11 </body>
12 </html>
```

Java Server Pages (JSP)

< > Java Server Pages (JSP)

JSP (JavaServer Pages) es una tecnología que permite incluir código Java en páginas web.

El denominado contenedor JSP (que sería un componente del servidor web) es el encargado de tomar la página, sustituir el código Java que contiene por el resultado de su ejecución, y enviarla al cliente. Así, se pueden diseñar fácilmente páginas con partes fijas y partes variables.



```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3
4 <!DOCTYPE html>
5 <html>
6 <head>
7   <meta charset="ISO-8859-1">
8   <title>Mostrar Fecha</title>
9 </head>
10 <body>
11   <%
12     java.util.Date d = new java.util.Date();
13     %>
14   <p><%=java.text.DateFormat.getDateInstance().format(d)%></p>
15 </body>
```

< > Java Server Pages (JSP)

¿Cómo funciona internamente JSP?

1. Los .jsp generan .java (servlets). Los servlets compilados son .class.
Es el contenedor de aplicaciones (tomcat, etc) el que convertirá los jsp en servlets.
2. El servidor leerá el archivo jsp. Luego, va a buscar el jsp compilado (el .class), si no lo encuentra, lo compila.
 - **Traducción** : pasar de jsp a java.
 - **Compilación** : pasar de java a .class



< > Java Server Pages (JSP)

Existen tres tipos de elementos JSP que podemos insertar en una página web:

- **Código** : podemos "incrustar" código Java de distintos tipos (declaraciones de variables y/o métodos, expresiones, sentencias) para que lo ejecute el contenedor JSP.
- **Directivas** : permiten controlar distintos parámetros del servlet resultante de la traducción automática del JSP
- **Acciones** : normalmente sirven para alterar el flujo normal de ejecución de la página (por ejemplo: redirecciones), aunque tienen usos variados.

Se pueden poner **comentarios** en una página JSP entre los símbolos `<%-- y --%>` . El contenedor JSP ignorará todo lo contenido entre ambos. Dentro de los fragmentos de código Java también se pueden colocar comentarios siguiendo la sintaxis habitual del lenguaje.



< > Java Server Pages (JSP)

Algunas ventajas de JSP:

1. **Fácil de mantener** : se puede administrar fácilmente porque podemos separar nuestra lógica de negocios con la lógica de presentación.
2. **Desarrollo rápido** : Si se modifica la página JSP, no es necesario volver a compilar ni implementar el proyecto.
3. **Menos código que Servlet:** En JSP, podemos usar muchas etiquetas, como etiquetas de acción, JSTL, etiquetas personalizadas, etc., que reducen el código.



LIVE DEMO

Creando un archivo .jsp:

Vamos a crear una página JSP llamada '**index.jsp**'. La página debe estar dentro de la carpeta '**webapp**' del proyecto.

Esta página debe tener en la etiqueta `<body>` debe incluir lo siguiente:

```
<body>
    <%    java.util.Date d = new java.util.Date();        %>
    <p><%=java.text.DateFormat.getDateInstance().format(d)%></p>
</body>
```



LIVE DEMO

A continuación, vamos a correr el proyecto en el servidor para que Tomcat muestre en el localhost lo que acabamos de codificar.

Deberíamos poder ver la fecha del día de hoy en el navegador determinado!

Tiempo: 20 minutos

Java Standard Tag Libs

JSTL

¿Qué es?

Es un conjunto de etiquetas (**tags**) standard que encapsulan funcionalidades de uso común para muchas aplicaciones con JSPs.

Debido a que las etiquetas JSTL son XML, se integran uniformemente con las etiquetas HTML y serán fáciles de usar por alguien que conozca html.

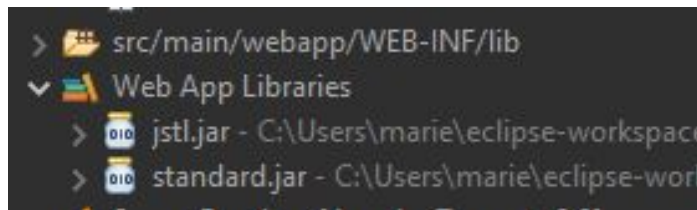
Las etiquetas JSTL están específicamente preparadas para realizar las tareas que van a tener lugar en la lógica de la presentación.



JSTL

Para utilizar las tags de JSTL es necesario cargar JSTL en nuestro proyecto:

Cargamos los archivos **jstl.jar** y **standard.jar** en la carpeta **lib** de nuestro proyecto. De esta forma no hará falta añadirlo al Java Build Path.



JSTL

Luego, incluimos la siguiente etiqueta en la cabecera de nuestro jsp, como vemos en la imagen

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```



```
index.jsp X
1  <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
2
3  <!DOCTYPE html>
4  <html>
5  <head>
```

Tag c:out



Tag `c:out`

¿Para qué sirve?:

Se utiliza para **imprimir o mostrar contenido en una página JSP**. Proporciona una forma segura de mostrar datos en la interfaz de usuario, evitando problemas de seguridad y ayudando a evitar ataques de inyección de scripts.

El tag `<c:out>` tiene la siguiente **sintaxis** básica:

`<c:out value="expresión" [atributos adicionales] />`

El atributo `value` especifica la expresión o variable cuyo valor se mostrará en la página JSP .



Tag `c:out`

Funcionalidad:

Además del atributo **value**, `<c:out>` admite **otros atributos** opcionales, como:

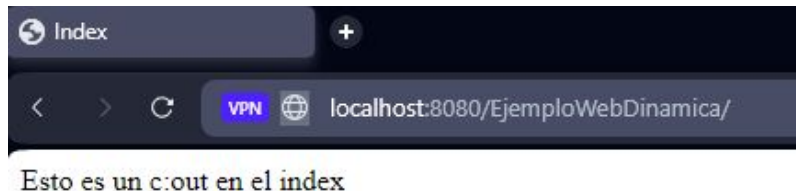
- **default**: Define un valor predeterminado que se mostrará si la expresión o variable tiene un valor nulo.
- **escapeXml**: Indica si se debe escapar o no los caracteres especiales de XML en el valor. El valor predeterminado es true.
- **charset**: Define el conjunto de caracteres que se utilizará para la salida. El valor predeterminado es el conjunto de caracteres de la página JSP.

Tag c:out

Veamos un ejemplo en código! En el archivo index, agregamos una tag c:out con un texto a mostrar:

```
3 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
4 <!DOCTYPE html>
5 <html>
6 <head>
10 <body>
11   <c:out value = "${'Esto es un c:out en el index'}" />
12 </body>
```

Si ejecutamos el proyecto en el servidor, podremos ver el mensaje en el navegador:



Tags $c : \text{if } y \ c : \text{choose}$



Tags `c:if` y `c:choose`

¿Para qué sirven?:

Se utilizan para realizar **toma de decisiones condicionales** en las páginas JSP. Estas etiquetas **permiten controlar el flujo de ejecución** en función de condiciones específicas.

El tag **<c:if>** se utiliza para evaluar una condición y ejecutar un bloque de código si la condición es verdadera.

Por otro lado, el tag **<c:choose>** se utiliza cuando se necesita realizar múltiples tomas de decisiones y ejecutar diferentes bloques de código en función de varias condiciones.



Tags `c:if` y `c:choose`

La etiqueta **c:if** tiene la siguiente sintaxis básica:

```
<c:if test="condición">  
  <!-- Código a ejecutar si la condición es verdadera -->  
</c:if>
```

El atributo **test** especifica la condición que se evaluará. Puede ser una expresión booleana, una variable booleana o una función que devuelve un valor booleano.

Si la condición se evalúa como verdadera, el bloque de código dentro de <c:if> se ejecutará.



Tags `c:if` y `c:choose`

Veamos un ejemplo práctico:

```
<c:set var="edad" value="25" />
```

```
<c:if test="${edad > 18}">
```

```
  <p>¡Eres mayor de edad!</p>
```

```
</c:if>
```

En este ejemplo, **`<c:set>`** se utiliza para asignar el valor "25" a la variable **`${edad}`**. Luego, **`<c:if>`** evalúa la condición **`${edad > 18}`** y muestra el mensaje **"¡Eres mayor de edad!"** si la condición es verdadera.

LIVE DEMO

Probando, probando:

En este ejemplo vamos a definir con `c:set` un valor para usuario y contraseña.

A continuación, vamos a validar con `c:if` si la contraseña es “12345”.

Si es válida, mostramos un mensaje por pantalla.

Tiempo: 15 minutos

Tag <c : choose>

Por otro lado, el tag **<c:choose>** puede contener múltiples etiquetas **<c:when>** y una etiqueta **<c:otherwise>**. Su sintaxis básica es la siguiente:

```
<c:choose>
  <c:when test="condición 1">
    <!-- Código a ejecutar si la condición1 es verdadera -->
  </c:when>
  <c:when test="condición 2">
    <!-- Código a ejecutar si la condición2 es verdadera -->
  </c:when>
  ...
  <c:otherwise>
    <!-- Código a ejecutar si ninguna de las condiciones anteriores es verdadera -->
  </c:otherwise>
</c:choose>
```



Tag `<c : choose>`

En cada etiqueta `<c:when>`, se **especifica una condición que se evaluará** . Si una condición es verdadera, se ejecutará el bloque de código correspondiente. **Si ninguna de las condiciones anteriores es verdadera** , se ejecutará el bloque de código dentro de `<c:otherwise>` .

Podemos compararlo con la estructura de control **Switch** en Java.

LIVE DEMO

Probando, probando:

En este ejemplo vamos a definir con `c:set` un valor para usuario y edad.

A continuación, vamos a validar con `c:choose` si la edad es:

- mayor a 10, mostraremos por pantalla “Es mayor a 10”
- mayor a 20, mostraremos por pantalla “Es mayor a 20”
- mayor a 50, mostraremos por pantalla “Es mayor a 50”
- Si no cumple ninguna, mostraremos “no cumple condición”

Tiempo: 15 minutos



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

JSP

JSP

Manos a la obra: 🙌

Ahora es tu turno de agregar un archivo 'index.jsp' al proyecto AlkeWallet.

Consigna: ✍️

- 1- Crear el archivo index.jsp en la carpeta webapp
- 2- Colocar 'Inicio' como título de pestaña
- 3- Colocar un párrafo que diga 'En este momento la hora es: '
- 4- Colocar una etiqueta jsp con el código Java necesario para mostrar la hora actual.

Tiempo 🕒: 20 minutos



Ejercicio N° 2

C:IF / C:CHOOSE

C:IF / C:CHOOSE

Manos a la obra: 🙌

Vamos a poner en práctica lo aprendido editando el archivo index.jsp del proyecto AlkeWallet.

Consigna: 📝

1. Crear una etiqueta c:if que valide si el usuario es mayor de edad (+18). En caso de que la respuesta sea verdadera, se debe dar la bienvenida al sistema con c:out.
2. Crear una etiqueta c:choose que defina qué día de la semana es y lo muestre por pantalla. Si es domingo, debe mostrar por pantalla un mensaje que diga “Día no hábil para realizar transacciones” con c:out.

Tiempo 🕒: 20 minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Comprender el concepto e implementación de JSP
- ✓ Reconocer el uso y configuración de JSTL
- ✓ Comprender la utilización de las etiquetas `c:out`, `c:if` y `c:choose`



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

